

Übungen: Hub & Spoke Netzwerktopologie

Azure Netzwerk-Grundlagen – VNet Peering ohne Firewall

Eigenständige Übung – keine Basis-VM nötig, reines Netzwerk-Lab

Für wen?	Alle Teilnehmer:innen – eigenständige Übung, unabhängig von den Hybrid-Cloud Übungen
Dauer	ca. 30–40 Min (+ optional 15–20 Min für den VM-Konnektivitätstest)
Ziel	Eine Hub-&-Spoke-Topologie aus drei VNets bauen und per Peering verbinden – und verstehen, warum Spoke1 und Spoke2 trotzdem nicht direkt miteinander reden können
Voraussetzung	Azure-Abo mit eigener Ressourcengruppe · Region frei wählbar (am besten die, die auch sonst genutzt wird)

Überblick – drei VNets, zwei Peerings

1 – VNets anlegen	Hub-VNet + zwei Spoke-VNets mit je einem Subnet erstellen	0 €
2 – Peering einrichten	Hub↔Spoke1 und Hub↔Spoke2 verbinden	0 € (Peering selbst ist kostenlos, nur Datenverkehr würde minimal kosten)
3 – Verifizieren	Peering-Status prüfen, Topologie im Portal ansehen	0 €
4 – VM Konnektivitätstest (<i>optional</i>)	Eine VM im Hub als Sprungbrett, Ping zu Spoke1/Spoke2 live testen	wenige Cent, wenn direkt danach gelöscht wird

i HINWEIS

Die Übung selbst (Schritt 1–6) braucht keine virtuellen Maschinen – VNets, Subnets und Peerings kosten nichts. Der VM-Test in Schritt 7–8 ist optional und nur dafür da, das Ergebnis live zu erleben statt nur im Portal zu lesen.

Zielarchitektur

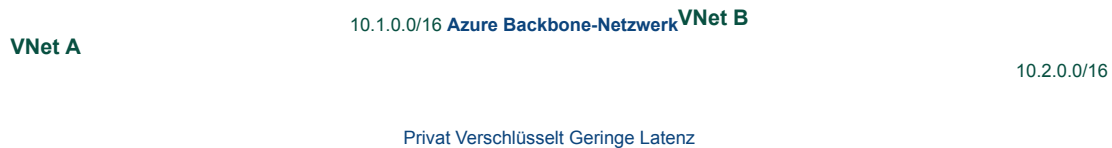


? WARUM Hub & Spoke?

In der Praxis liegen zentrale Dienste (z. B. Firewall, DNS, VPN-Gateway) im Hub, während einzelne Workloads (Abteilungen, Projekte, Umgebungen) jeweils in einem eigenen Spoke isoliert sind. Jeder Spoke ist mit dem Hub verbunden, aber nicht automatisch mit den anderen Spokes – das hält die Umgebung übersichtlich und sicher.

In diesem Lab bauen wir nur das Grundgerüst; eine Firewall im Hub käme später dazu. **Was ist**

VNet Peering eigentlich?



Peering verbindet zwei VNets direkt über das private Microsoft-Backbone – der Traffic geht nie über das öffentliche Internet. Wichtig fürs Verständnis: Peering ist **nicht transitiv**. Wenn A mit B gepeert ist und B mit C, heißt das nicht, dass A auch mit C reden kann. Genau das bauen wir gleich nach.

Schritt 1 – Ressourcengruppe erstellen

Azure Portal → „Ressourcengruppen“ → „+ Erstellen“
Name z. B. rg-hubspoke-lab
Region frei wählbar – am besten dieselbe wie in den anderen Übungen

Schritt 2 – Hub-VNet erstellen

Azure Portal → „Virtuelle Netzwerke“ suchen → „+ Erstellen“
Ressourcengruppe: rg-hubspoke-lab , Name: vnet-hub
Reiter „IP-Adressen“: Adressraum 10.0.0.0/16 , ein Subnet 10.0.0.0/24
Überprüfen + erstellen

Schritt 3 – Spoke-VNets erstellen

Gleiches Vorgehen zweimal wiederholen:
vnet-spoke1 mit Adressraum 10.1.0.0/16 , Subnet 10.1.0.0/24
vnet-spoke2 mit Adressraum 10.2.0.0/16 , Subnet 10.2.0.0/24

i HINWEIS

Die drei Adressräume dürfen sich nicht überschneiden – sonst lässt sich später kein Peering einrichten. Deshalb bewusst 10.0.x, 10.1.x, 10.2.x verwenden.

Schritt 4 – Peering Hub ↔ Spoke1

Im Portal: vnet-hub öffnen → „Peerings“ → „+ Hinzufügen“
Name der Verbindung von Hub aus: hub-to-spoke1
Remote-VNet: vnet-spoke1
Name der Verbindung von Spoke1 aus: spoke1-to-hub
Standardeinstellungen übernehmen (Traffic zulassen, Weiterleitung aus, Gateway-Transit aus) → Hinzufügen

Schritt 5 – Peering Hub ↔ Spoke2

Gleicher Ablauf wie Schritt 4, diesmal zwischen vnet-hub und vnet-spoke2
Verbindungsnamen z. B. hub-to-spoke2 und spoke2-to-hub

i HINWEIS

Ein Peering entsteht in Azure immer als zwei Verbindungen – eine pro Richtung. Beide müssen den Status „Verbunden“ haben, sonst funktioniert die Verbindung nur einseitig oder gar nicht.

Schritt 6 – Verifizieren

Bei `vnet-hub` → „Peerings“: zwei Einträge mit Status „Verbunden“

Bei `vnet-spoke1` und `vnet-spoke2` → „Peerings“: je ein Eintrag mit Status „Verbunden“ Optional:

„Diagramm“-Ansicht des Hub-VNets öffnen – zeigt die Topologie grafisch

✓ SO SIEHT ERFOLG AUS

Alle drei VNets zeigen unter „Peerings“ den Status „Verbunden“

Die Diagramm-Ansicht des Hub-VNets zeigt beide Spokes als verbundene Knoten

Spoke1 und Spoke2 tauchen bei sich gegenseitig nicht als Peering auf

Optional: Ping von `vm-hub` zu `vm-spoke1` und `vm-spoke2` erfolgreich, Ping von `vm-spoke1` zu `vm-spoke2` läuft in ein Timeout

Optional – live erleben statt nur nachlesen

Für den Ping-Test reicht eine Public IP auf der Hub-VM völlig aus – sie dient als „Sprungbrett“ (Jump-Host). Die Spoke-VMs bleiben komplett privat und sind trotzdem über das Peering erreichbar, genau wie in echten Hub-&Spoke-Umgebungen üblich.

Dreimal „Virtuelle Maschine erstellen“: `vm-hub` (in `vnet-hub`), `vm-spoke1` (in `vnet-spoke1`), `vm-spoke2` (in `vnet-spoke2`)

Image: Ubuntu Server (aktuelle LTS-Version), Größe: B1s

Authentifizierung: SSH-Schlüssel (Standard) oder Passwort, je nachdem was gewohnt ist Nur bei

`vm-hub` : Öffentliche IP-Adresse: Ja · Eingehende Ports: SSH (22) zulassen Bei `vm-spoke1` und

`vm-spoke2` : Öffentliche IP-Adresse: Nein

Bei „Netzwerk“ jeweils das passende VNet/Subnet auswählen (nicht das Standard-VNet übernehmen!)

i HINWEIS

Bei Linux-VMs ist ICMP (Ping) standardmäßig nicht blockiert, anders als bei Windows – der Ping-Test funktioniert also ohne zusätzliche Firewall-Konfiguration auf der VM, solange das Peering steht.

Konnektivität testen

Private IP-Adressen von `vm-spoke1` und `vm-spoke2` im Portal nachsehen (Übersicht → „Private IP-Adresse“) Per SSH auf `vm-hub` verbinden (öffentliche IP von `vm-hub` verwenden)

Von dort aus die private IP von `vm-spoke1` anpingen → sollte antworten

Von dort aus die private IP von `vm-spoke2` anpingen → sollte ebenfalls antworten

Von `vm-hub` aus per SSH weiter auf `vm-spoke1` springen (private IP), von dort die private IP von `vm-spoke2` anpingen → sollte **fehlschlagen** (Timeout)

Das ist der eigentliche Lernmoment der Übung: Hub erreicht beide Spokes, aber die Spokes erreichen sich untereinander nicht – Peering ist nicht transitiv.

⚠ Problem: Peering lässt sich nicht erstellen, Fehler zu Adressräumen

→ Lösung: Zwei VNets haben überlappende Adressräume. Adressraum eines der beiden VNets korrigieren (z. B. im Subnet-Bereich anpassen) und erneut versuchen.

⚠ Problem: Peering zeigt „Nicht verbunden“ oder „Getrennt“

→ Lösung: Meist wurde nur eine Richtung angelegt. Im jeweils anderen VNet unter „Peerings“ prüfen, ob die Gegenverbindung existiert, notfalls das Peering löschen und neu einrichten.

⚠ Problem: Spoke1 kann Spoke2 trotzdem nicht erreichen (auch wenn beide mit dem Hub gepeert sind)

→ Das ist kein Fehler, sondern erwartetes Verhalten: Peering ist nicht transitiv. Für Spoke-zu-Spoke-Verkehr bräuchte man zusätzlich eine Route über eine Firewall/NVA im Hub – das ist Thema einer späteren Stunde.

⚠ Problem (optional): Ping funktioniert auch innerhalb eines erfolgreichen Peerings nicht (z. B. Hub → Spoke1)

→ Lösung: Meist blockt die Standard-NSG der Spoke-VM eingehenden Traffic nicht, aber falls doch angepasst wurde: in der Netzwerksicherheitsgruppe der Spoke-VM eine Inbound-Regel für ICMP von VirtualNetwork zulassen.

AUFRÄUMEN (nicht vergessen!)

Azure Portal → Ressourcengruppe rg-hubspoke-lab öffnen → „Ressourcengruppe löschen“ Das entfernt alle drei VNets und beide Peerings in einem Schritt. Falls die optionale VM-Übung gemacht wurde, sind alle drei VMs damit ebenfalls gelöscht.